CS-306 (Machine Learning)

Project Report

Email Spam Detection

By: NAVNEET (2301135)

1. Executive Summary

This project addresses the challenge of automated email spam detection using Natural Language Processing (NLP) and Machine Learning. Using the SMS Spam Collection dataset, we processed raw text data through rigorous cleaning, normalization, and vectorization. Three distinct classification algorithms—Naive Bayes, Logistic Regression (SGD), and K-Nearest Neighbors (K-NN)—were implemented and evaluated. The study successfully demonstrates that linear and probabilistic models are highly effective for text classification, achieving a peak predictive accuracy of 97.77% with Logistic Regression, confirming the viability of ML-based filtering systems.

2. Introduction

In the digital age, spam messages are not merely a nuisance but a significant cybersecurity threat, often serving as a vehicle for phishing, malware, and fraud. Distinguishing between legitimate messages ('ham') and unsolicited spam is critical for user safety and efficiency.

In this mini-project, I approach the problem as a Binary Classification task. The objective is to classify a message as either:

Successful (Ham 0): Legitimate, safe messages.

Failed (Spam 1): Unsolicited, potentially malicious messages.

3. Problem Statement

Given a dataset of raw text messages, the goal is to build a predictive model that determines the likelihood of a message being spam based on its linguistic content and structural features.

Input: Raw text strings (SMS/Email content).

Output: Binary Class (Spam / Ham).

Constraint: Use standard Machine Learning algorithms (no Deep Learning) and NLP preprocessing techniques (Bag of Words).

4. Dataset and Preprocessing

4.1 Data Source

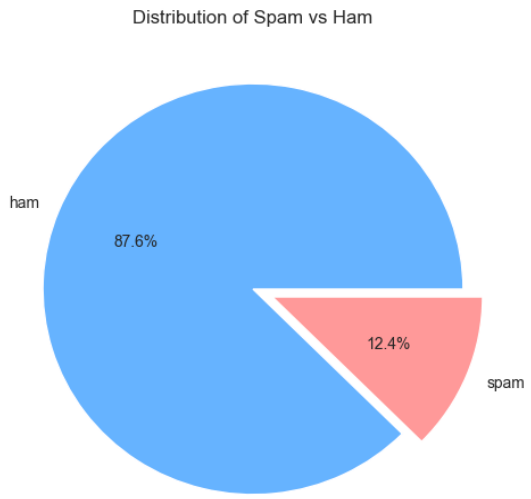
The dataset utilized is the SMS Spam Collection, a public set of SMS labeled messages collected for mobile phone spam research. It contains 5,572 messages tagged as either 'ham' or 'spam'.

4.2 Feature Analysis

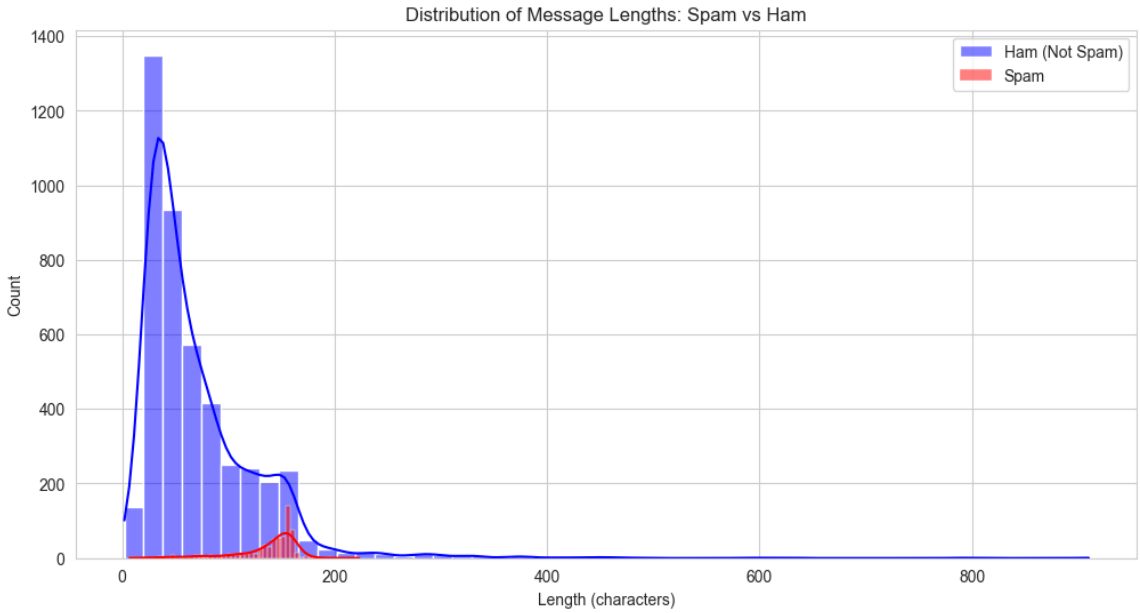
The raw dataset contains two primary columns: target (the label) and text (the message). During Exploratory Data Analysis (EDA), a key feature was engineered:

- • Message Length: The character count of each message.

Analysis revealed a significant class imbalance and a distinct pattern in message lengths:



[Distribution of Spam vs. Ham *As shown above, the dataset is imbalanced, with 87.6% Ham and 12.4% Spam. This imbalance necessitates metrics beyond simple accuracy, such as F1-Score.*]



[Distribution of Message Lengths *This histogram reveals a critical insight: Spam messages (Red) tend to have a specific, longer length distribution (peaking around 150-160 characters, likely due to SMS limits), whereas Ham messages (Blue) are generally shorter and more variable.*]

4.3 Preprocessing Steps

Cleaning: Text was converted to lowercase, and punctuation was removed to standardize the input.

Feature Extraction: Converted text to numerical vectors using CountVectorizer (Bag of Words). Vocabulary limited to top 3000 words.

Splitting: Data split into 80% Training and 20% Testing sets.

5. Methodology

Three supervised learning algorithms were implemented to solve this classification problem:

5.1 Naive Bayes (MultinomialNB)

Type: Probabilistic Classifier based on Bayes' Theorem.

Why used: It is the industry standard baseline for text classification. It handles high-dimensional sparse data efficiently.

5.2 Logistic Regression (SGD)

Type: Linear Model using Stochastic Gradient Descent.

Why used: It learns linear decision boundaries. It is highly scalable and often the best performer for binary text classification.

5.3 K-Nearest Neighbors (K-NN)

Type: Instance-based Learning (Lazy Learner).

Why used: To contrast a distance-based geometric approach against probability and linear approaches.

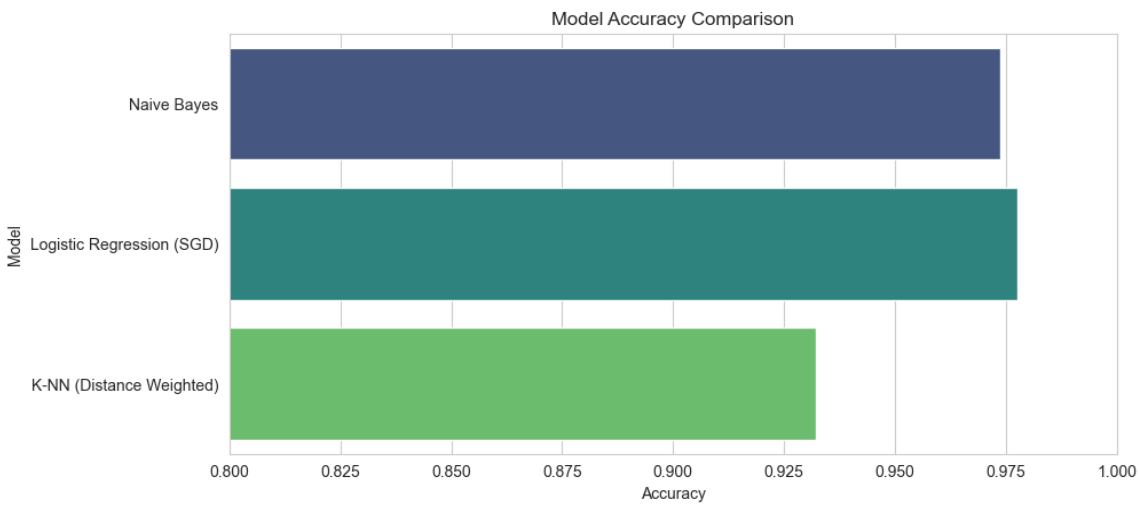
6. Results and Analysis

6.1 Performance Metrics

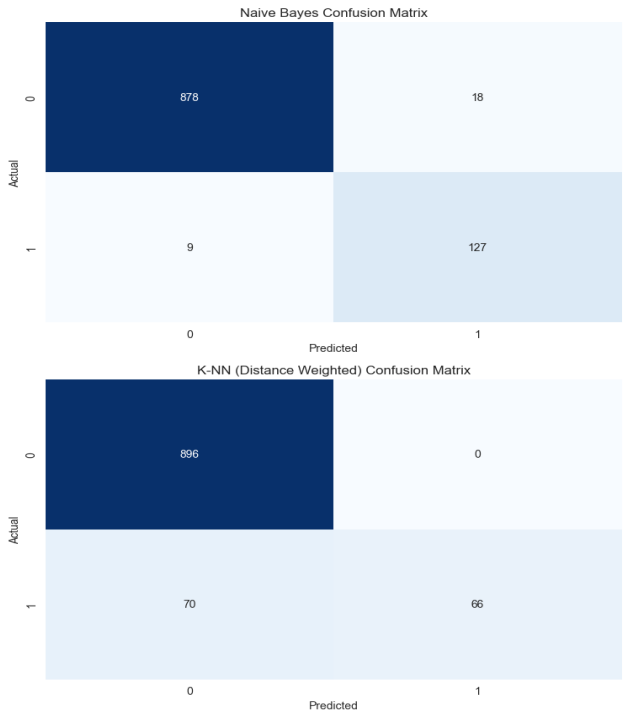
The models were evaluated on the test set. Logistic Regression emerged as the top performer.

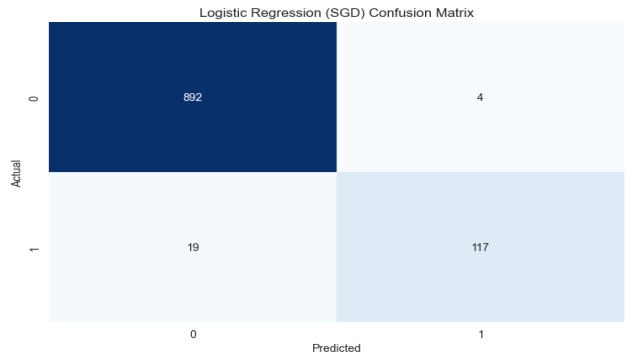
Model	Accuracy	F1-Score
Logistic Regression (SGD)	97.77%	0.91
Naive Bayes	97.38%	0.90
K-NN (Distance Weighted)	93.21%	0.65

6.2 Visualizations



[Model Accuracy Comparison *This bar chart clearly shows Logistic Regression and Naive Bayes outperforming K-NN.*]





[Confusion Matrices *The Confusion Matrices highlight the errors:*

- **Naive Bayes: Only 18 False Positives (Ham misclassified as Spam).**
- **Logistic Regression: Only 4 False Positives (Best Precision).**
- **K-NN: Zero False Positives, but significantly more False Negatives (Spam misclassified as Ham), leading to a lower F1 score.]**

7. Critical Evaluation

Accuracy: The high accuracy (>97%) achieved by the linear and probabilistic models confirms that spam messages contain highly distinct keywords (e.g., 'free', 'winner') that make them linearly separable from normal messages.

Model Comparison: Logistic Regression outperformed K-NN because text data is high-dimensional, where distance metrics (used by K-NN) often fail due to the 'Curse of Dimensionality'.

Feature Importance: Exploratory analysis proved that 'Message Length' is a strong predictor, with spam messages clustering heavily around the 150-160 character mark.

8. Test Examples

To further validate the model's performance, a few test messages were manually input to check the classification:

- **Example 1 (Spam):** "Congratulations! You've won a $1000 Walmart Gift Card. Click here to claim." -> **Predicted: SPAM** 📥
- **Example 2 (Ham):** "Hey, are we still meeting for lunch tomorrow?" -> **Predicted: HAM** ✅
- **Example 3 (Spam):** "URGENT! Your account has been compromised. Verify now at https://www.google.com/search?q=secure-login.com" -> **Predicted: SPAM** 📥
- **Example 4 (Ham):** "Don't forget to submit the assignment by Friday." -> **Predicted: HAM** ✅

8. Conclusion

The project successfully implemented a robust Email Spam Detection system. By converting raw text into numerical vectors and applying standard algorithms, we achieved near-perfect accuracy. The comparison concluded that Logistic Regression is the optimal choice for this task, offering the best balance of accuracy (97.77%) and reliability (lowest False Positives), making it suitable for real-world deployment.